```
class GridWorldMDP:
    def __init__(self):
        self.grid_shape = (2, 3)
        self.states = ['S0', 'S1', 'G', 'S3', 'S4']
        self.actions = ['Up', 'Down', 'Left', 'Right']
        self.gamma = 0.8

        self.grid_map = {
            'S0': (0, 0),
            'S1': (0, 1),
            'G': (0, 2),
            'X': (1, 0),
            'S3': (1, 1),
            'S4': (1, 2)
        }

        self.transitions = {
            ('S0', 'Up'): 'S0',
            ('S0', 'Down'): 'S0',
            ('S0', 'Left'): 'S0',
            ('S0', 'Right'): 'S1',
            ('S1', 'Up'): 'S1',
            ('S1', 'Down'): 'S3',
            ('S1', 'Left'): 'S0',
            ('S1', 'Right'): 'G',
            ('S3', 'Up'): 'S1',
            ('S3', 'Down'): 'S3',
            ('S3', 'Left'): 'S3',
            ('S3', 'Right'): 'S4',
            ('S4', 'Up'): 'G',
            ('S4', 'Down'): 'S4',
            ('S4', 'Left'): 'S3',
            ('S4', 'Right'): 'S4'
        }

    def transition_function(self, state, action):
        if state == 'G':
            return 'G'
        return self.transitions.get((state, action), state)

    def reward_function(self, next_state):
        if next_state == 'G':
            return 10.0
        elif next_state == 'X':
            return -5.0
        else:
            return -1.0


def calculate_returns():
    print("--- ASSIGNMENT 1: UNDISCOUNTED & DISCOUNTED RETURNS ---")
    n = int(input("Enter number of time steps/episodes: "))
    rewards = []
    for i in range(n):
        r = float(input(f"Enter reward for time step {i + 1}: "))
        rewards.append(r)

    gamma = float(input("Enter discount factor (gamma): "))

    undiscounted_return = sum(rewards)

    discounted_return = 0.0
    for t, r in enumerate(rewards):
        discounted_return += (gamma ** t) * r

    print("Entered reward values:", rewards)
    print("Undiscounted return:", undiscounted_return)
    print("Discounted return:", discounted_return)
    print()


def run_mdp_demo():
    print("--- ASSIGNMENT 2: 2x3 GRID WORLD MDP ---")
    env = GridWorldMDP()

    state_space = env.states
    print("a. State Space:", state_space)

    action_space = env.actions
    print("b. Action Space:", action_space)
```

```
    print("c. Reward Function Demonstration:")
    for state in ['S0', 'G', 'X']:
        print(f"   Reward for reaching {state}: {env.reward_function(state)}")

    discount_factor = env.gamma
    print("d. Discount Factor:", discount_factor)

    current_state = 'S1'
    action = 'Right'
    next_state = env.transition_function(current_state, action)
    print("e. Transition Function defined in GridWorldMDP class.")

    print(f"f. Next state from {current_state} with action '{action}': {next_state}")


if __name__ == "__main__":
    calculate_returns()
    run_mdp_demo()
```

```
--- ASSIGNMENT 1: UNDISCOUNTED & DISCOUNTED RETURNS ---
Enter number of time steps/episodes: 3
Enter reward for time step 1: -1
Enter reward for time step 2: -1
Enter reward for time step 3: 10
Enter discount factor (gamma): 0.8
Entered reward values: [-1.0, -1.0, 10.0]
Undiscounted return: 8.0
Discounted return: 4.600000000000001

--- ASSIGNMENT 2: 2x3 GRID WORLD MDP ---
a. State Space: ['S0', 'S1', 'G', 'S3', 'S4']
b. Action Space: ['Up', 'Down', 'Left', 'Right']
c. Reward Function Demonstration:
   Reward for reaching S0: -1.0
   Reward for reaching G: 10.0
   Reward for reaching X: -5.0
d. Discount Factor: 0.8
e. Transition Function defined in GridWorldMDP class.
f. Next state from S1 with action 'Right': G
```